VULNERABILITY ASSESSMENT REPORT

PASSIVE WEB APPLICATION SECURITY ASSESSMENT
CONDUCTED USING OWASP ZAP

PREPARED BY :
LAIKEN NAIDOO

CYBER SECURITY INTERNSHIP
FEBRUARY 2026

TABLE OF CONTENTS

EXECUTIVE SUMMARY

This Vulnerability Assessment was conducted on the publicly accessible website **testphp.vulnweb.com** using ethical, read-only techniques. The objective of this assessment was to identify common web security weaknesses, classify risks in business-friendly language, and provide practical remediation recommendations.

The assessment was performed using OWASP ZAP (Passive Mode) along with controlled manual exploration of the website. No exploitation, brute force attacks, login bypass attempts, or denial-of-service techniques were performed during this assessment.

A total of 12 findings were identified:

- 0 High Risk
- 3 Medium Risk
- 5 Low Risk
- 4 Informational

No critical vulnerabilities were detected. However, several Medium-risk security configuration gaps were identified that may increase exposure to attacks such as Cross-Site Scripting (XSS), clickjacking, and unauthorized request execution if left unaddressed.

Overall, the website's security posture can be classified as Moderate Risk. Immediate priority should be given to addressing the identified Medium-risk findings.

SCOPE & METHODOLOGY

Scope

This Vulnerability Assessment was conducted on the publicly accessible website:
http://testphp.vulnweb.com

The scope of this assessment was limited to publicly available pages and client-side components accessible without authentication.
The assessment was performed using strictly passive and non-intrusive techniques. No login attempts, brute force attacks, denial-of-service activity, exploitation attempts, or modification of data were performed during this engagement.

This assessment focused on identifying:

- Security header misconfigurations
- Cookie security weaknesses
- Information leakage through HTTP headers
- Missing security controls
- Potential client-side exposure risks

The objective was to simulate how an external attacker may observe the application without actively exploiting vulnerabilities.
The assessment did not include authenticated testing or internal infrastructure review.

Methodology

The assessment was conducted using **OWASP ZAP (Zed Attack Proxy)** in Passive Mode. The target website was manually explored through a proxied browser session to allow passive analysis of HTTP responses, headers, cookies, and client-side components.

The following approach was followed:

- Manual browsing of publicly accessible pages
- Passive crawling and analysis of application responses
- Review of HTTP security headers
- Inspection of cookie attributes (HttpOnly, SameSite, Secure)
- Identification of information leakage through server response headers

No active scanning modules were enabled during this assessment. The objective was to identify observable security weaknesses without interacting destructively with the application.
Findings were classified according to OWASP ZAP's risk levels: High, Medium, Low, and Informational.

Tools Used

- **OWASP ZAP (Zed Attack Proxy)**

Used in Passive Mode to analyze HTTP responses, security headers, cookies, and application behavior without performing active exploitation.

- **Web Browser (Firefox, Proxied Through ZAP)**

Used for controlled manual exploration of publicly accessible pages to trigger passive analysis.

No automated active scanning tools, exploitation frameworks, or intrusive techniques were used during this engagement.

Summary of Findings

The following vulnerabilities and security observations were identified during the passive assessment of the target website:

Risk Distribution Overview
- High Risk: 0
- Medium Risk: 3
- Low Risk: 5
- Informational: 4

Medium Risk Findings (Priority)
- Absence of Anti-CSRF Tokens
- Content Security Policy (CSP) Header Not Set
- Missing Anti-Clickjacking Header

These findings may increase exposure to client-side attacks such as Cross-Site Scripting (XSS), clickjacking, and unauthorized request execution.

Low Risk Findings
- Cookie No HttpOnly Flag
- Cookie without SameSite Attribute
- Server Leaks Information via "X-Powered-By" Header
- Server Leaks Version Information via "Server" Header
- X-Content-Type-Options Header Missing

These findings indicate security hardening improvements that should be addressed to reduce information disclosure and strengthen session protection.

Informational Findings
- Modern Web Application Identified
- Session Management Response Identified
- Charset Mismatch (Header vs Meta Content-Type)
- User Controllable HTML Element Attribute (Potential XSS)

These findings provide contextual information about the application structure and behavior but do not represent immediate security threats.

Detailed Findings

1. Content Security Policy (CSP) Header Not Set
Risk Level: **Medium**

Description
The assessment identified that the Content Security Policy (CSP) HTTP response header is not configured on the target website. A CSP header helps define which content sources are permitted to load within the browser, thereby reducing the impact of certain client-side attacks such as Cross-Site Scripting (XSS).
Without a properly configured CSP, the application may be more susceptible to malicious script injection and content manipulation attacks.
Impact

An attacker who successfully injects malicious scripts into the application may execute arbitrary JavaScript within a user's browser. This could potentially lead to:

- Session hijacking
- Credential theft
- Unauthorized actions performed on behalf of users
- Defacement or malicious content injection

Although CSP does not prevent vulnerabilities directly, it acts as an important security control that limits exploit impact.

Evidence
OWASP ZAP Passive Scan Alert:
"Content Security Policy (CSP) Header Not Set"
URL: **http://testphp.vulnweb.com/**

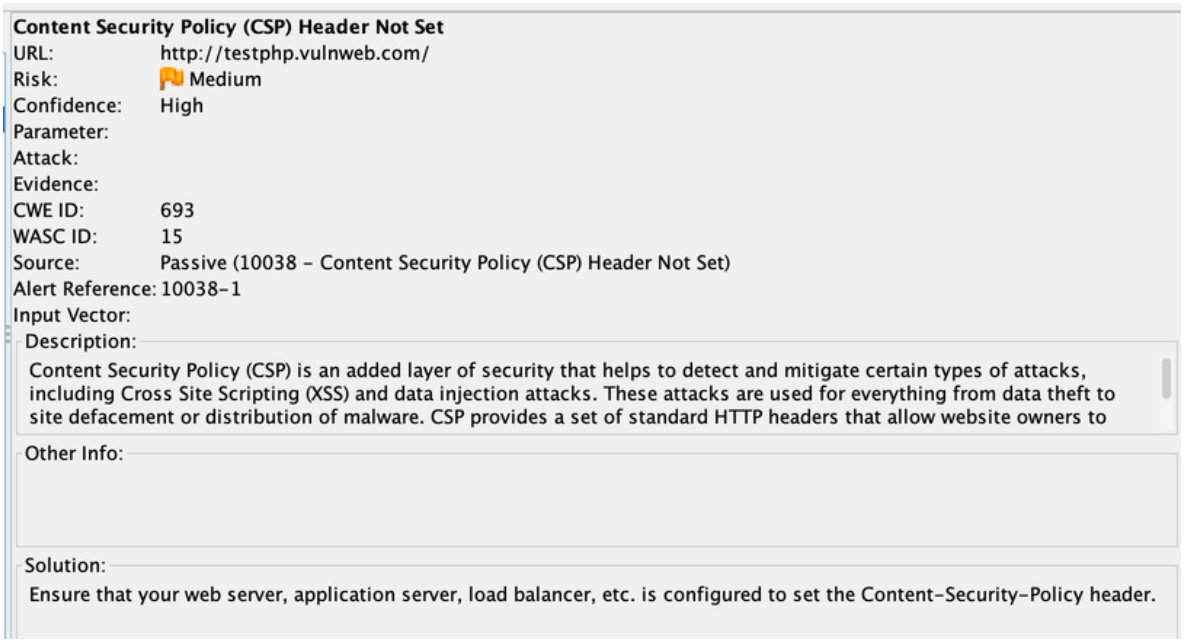

Figure 1: OWASP ZAP Alert – CSP Header Not Set

Recommendation
Configure and implement a properly defined Content-Security-Policy HTTP response header. The policy should restrict script sources, object sources, and other dynamic content to trusted domains only.

2. Absence of Anti-CSRF Tokens (Systemic)
Risk Level: **Medium**

Description
The assessment identified that the application does not implement Anti-CSRF (Cross-Site Request Forgery) tokens in forms or state-changing requests.
CSRF tokens are security mechanisms designed to ensure that requests submitted to the application originate from legitimate users and not from malicious third-party websites.
Without CSRF protection, an attacker could potentially trick an authenticated user into performing unintended actions within the application without their knowledge.

Impact
If exploited, this weakness may allow:
• Unauthorized actions performed on behalf of a logged-in user
• Forced form submissions
• Modification of user data
• Transaction manipulation

This type of vulnerability can compromise user trust and data integrity.

Evidence
OWASP ZAP Passive Scan Alert:
"Absence of Anti-CSRF Tokens (Systemic)"
URL: **http://testphp.vulnweb.com**

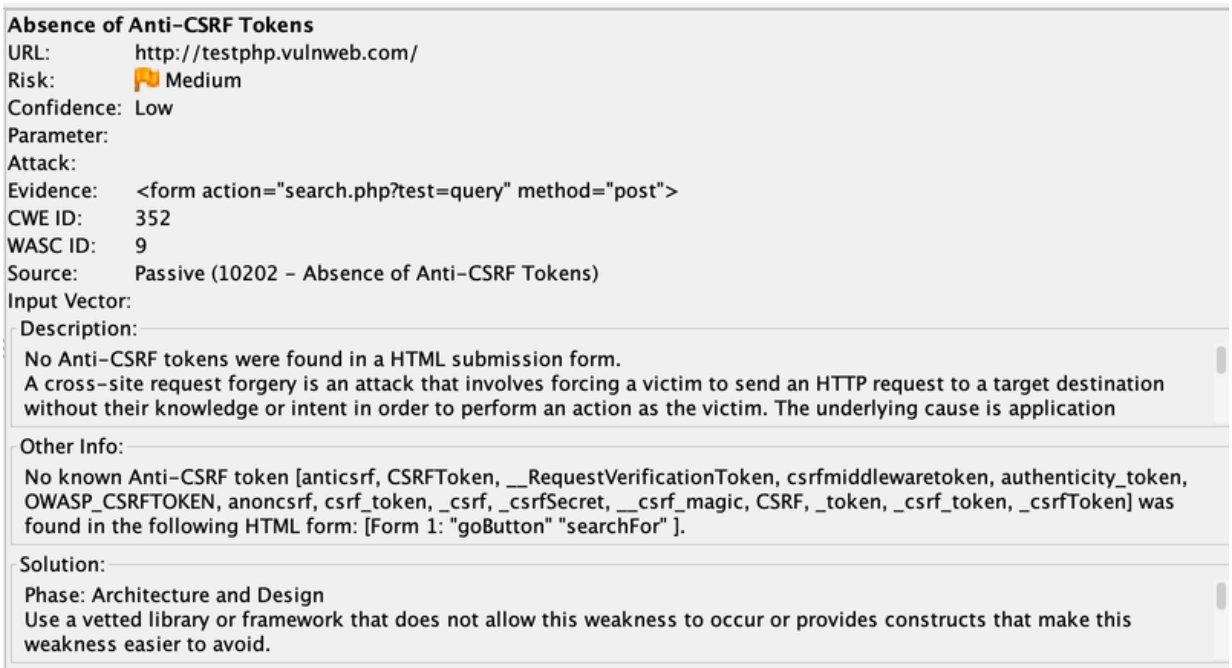

Figure 2: OWASP ZAP Alert – Absence of Anti-CSRF Tokens

Recommendation
Implement Anti-CSRF tokens in all forms and state-changing requests.
Each request should include a unique, unpredictable token generated server-side and validated upon submission.
Framework-level CSRF protection mechanisms (if available) should be enabled to ensure consistent enforcement across the application.

3. Missing Anti-Clickjacking Header (Systemic)

Risk Level: **Medium**

Description

The assessment identified that the application does not implement anti-clickjacking protections such as the X-Frame-Options or Content-Security-Policy frame-ancestors header.

Clickjacking occurs when a malicious website embeds the target application inside an invisible or deceptive frame, tricking users into clicking on elements they did not intend to interact with.

Without anti-clickjacking headers, the application may be rendered within an attacker-controlled page.

Impact

If exploited, this weakness may allow:

• Unauthorized actions triggered through deceptive user interaction

• Credential compromise

• Transaction manipulation

• Reputational damage

This increases exposure to social engineering and UI redressing attacks.

Evidence

OWASP ZAP Passive Scan Alert:

“Missing Anti-Clickjacking Header (Systemic)”

URL: **http://testphp.vulnweb.com**

Figure 3: OWASP ZAP Alert – Missing Anti-Clickjacking Header

Recommendation

Implement clickjacking protection using one of the following:

• Configure the X-Frame-Options HTTP header (e.g., DENY or SAMEORIGIN)

OR

• Define the frame-ancestors directive within a properly configured Content-Security-Policy header

This will prevent the application from being embedded within unauthorized external websites.

4. Low Risk Findings

The following issues were identified during the passive assessment and are classified as Low Risk. While they do not represent immediate critical threats, they indicate areas where security hardening should be improved.
Although classified as Low Risk, addressing these issues contributes to overall defense-in-depth strategy.

4.1 Cookie Without HttpOnly Flag

Risk Level: **Low**

Description:
The application sets cookies without the HttpOnly attribute enabled.
The HttpOnly flag prevents client-side scripts (such as JavaScript) from accessing cookie data. Without this flag, cookies may be exposed if a Cross-Site Scripting (XSS) vulnerability exists.

Recommendation:
Configure all session and authentication cookies with the HttpOnly attribute to reduce the risk of session theft via client-side script access.

4.2 Cookie Without SameSite Attribute

Risk Level: **Low**

Description:
The application does not define the SameSite attribute for cookies.
The SameSite attribute helps prevent cookies from being sent in cross-site requests, reducing the risk of Cross-Site Request Forgery (CSRF) attacks.

Recommendation:
Configure cookies with an appropriate SameSite policy (e.g., Lax or Strict) depending on application requirements.

4.3 Server Leaks Information via “X-Powered-By” Header

Risk Level: **Low**

Description:
The server discloses technology stack details through the “X-Powered-By” HTTP response header.
Revealing backend technology versions may assist attackers in identifying known vulnerabilities.

Recommendation:
Disable or suppress the “X-Powered-By” header within server configuration.

4.4 Server Leaks Version Information via “Server” Header
Risk Level: Low

Description:
The “Server” HTTP response header reveals version information about the web server software.
This information may assist attackers in targeting specific exploits.

Recommendation:
Configure the web server to limit or suppress version disclosure.

4.5 X-Content-Type-Options Header Missing
Risk Level: Low

Description:
The application does not set the X-Content-Type-Options header.
Without this header, browsers may attempt MIME-type sniffing, which could expose the application to content-type confusion attacks.

Recommendation:
Implement the header:
X-Content-Type-Options: nosniff

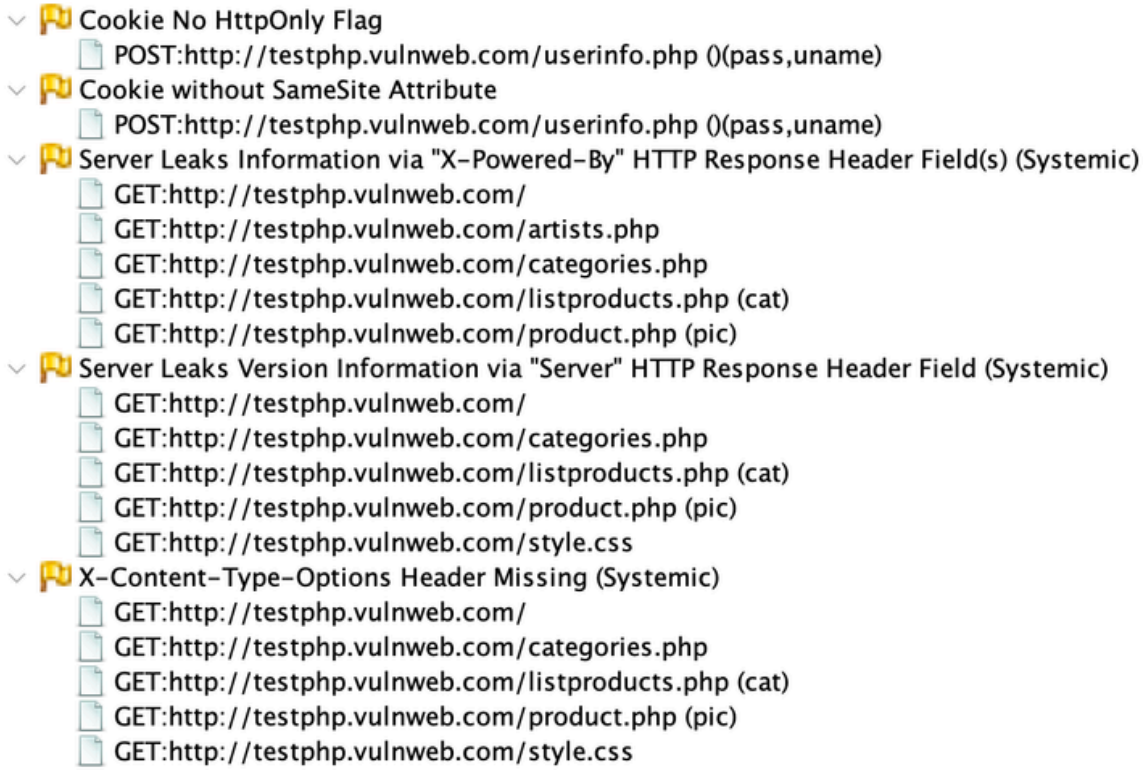

Figure 4: OWASP ZAP Low Risk Alerts Overview

5. Informational Findings

The following informational alerts were identified during the passive assessment. These findings do not represent immediate security vulnerabilities but provide contextual insights into the application's structure and configuration.

5.1 Modern Web Application Identified

The application utilizes modern client-side scripting techniques and dynamic content loading mechanisms.
This observation does not indicate a vulnerability but suggests that client-side security controls and headers should be properly configured.

5.2 Session Management Response Identified

Session management behavior was observed in HTTP responses.
While not inherently insecure, session cookies should be configured securely with appropriate attributes such as Secure, HttpOnly, and SameSite.

5.3 Charset Mismatch (Header vs Meta Content-Type)

A mismatch between character encoding defined in HTTP headers and HTML meta tags was observed.
Although typically low impact, inconsistent encoding configurations may lead to unexpected rendering or processing behavior.

5.4 User Controllable HTML Element Attribute (Potential XSS)

User-controlled input was identified within HTML attributes.
No exploitation was performed; however, proper input validation and output encoding should be maintained to prevent potential Cross-Site Scripting (XSS) risks.

Conclusion

This Vulnerability Assessment was conducted using passive, read-only techniques in accordance with ethical security testing principles.
No critical or high-risk vulnerabilities were identified during this engagement. However, several medium and low-risk configuration weaknesses were observed, primarily related to security headers, cookie attributes, and client-side protections.
Addressing the identified Medium-risk findings — including the absence of Content Security Policy (CSP), Anti-CSRF protections, and anti-clickjacking headers — should be prioritized to strengthen the application's overall security posture.
Low-risk configuration issues should also be remediated as part of standard security hardening practices.
Overall, the application demonstrates moderate security maturity but would benefit from improved implementation of standard web security controls, secure cookie configurations, and comprehensive HTTP response header protections.